

## 第49章 在 SRAM 中调试代码

本章参考资料：《STM32F4xx 中文参考手册》、《STM32F4xx 规格书》、《Cortex-M3 权威指南》、《Cortex-M4 Technical Reference Manual》(跟 M3 大部分是相同的，读英文不习惯可先参考《Cortex-M3 权威指南》)。

学习本章时，配合《STM32F4xx 中文参考手册》“存储器和总线结构”及“嵌入式 FLASH 接口”章节一起阅读，效果会更佳，特别是涉及到寄存器说明的部分。

### 49.1 在 RAM 中调试代码

一般情况下，我们在 MDK 中编写工程应用后，调试时都是把程序下载到芯片的内部 FLASH 运行测试的，代码的 CODE 及 RW-data 的内容被写入到内部 FLASH 中存储。但在某些应用场合下却不希望或不能修改内部 FLASH 的内容，这时就可以使用 RAM 调试功能了，它的本质是把原来存储在内部 FLASH 的代码(CODE 及 RW-data 的内容)改为存储到 SRAM 中(内部 SRAM 或外部 SRAM 均可)，芯片复位后从内部 SRAM 中加载代码并运行。

把代码下载到 RAM 中调试有如下优点：

- ❑ 下载程序非常快。RAM 存储器的写入速度比在内部 FLASH 中要快得多，且没有擦除过程，因此在 RAM 上调试程序时程序几乎是秒下的，对于需要频繁改动代码的调试过程，能节约很多时间，省去了烦人的擦除与写入 FLASH 过程。另外，STM32 的内部 FLASH 可擦除次数为 1 万次，虽然一般的调试过程都不会擦除这么多次导致 FLASH 失效，但这确实也是一个考虑使用 RAM 的因素。
- ❑ 不改写内部 FLASH 的原有程序。
- ❑ 对于内部 FLASH 被锁定的芯片，可以把解锁程序下载到 RAM 上，进行解锁。
- ❑ 相对地，把代码下载到 RAM 中调试有如下缺点：
- ❑ 存储在 RAM 上的程序掉电后会丢失，不能像 FLASH 那样保存。
- ❑ 若使用 STM32 的内部 SRAM 存储程序，程序的执行速度与在 FLASH 上执行速度无异，但 SRAM 空间较小。
- ❑ 若使用外部扩展的 SRAM 存储程序，程序空间非常大，但 STM32 读取外部 SRAM 的速度比读取内部 FLASH 慢，这会导致程序总执行时间增加，因此在外 SRAM 中调试的程序无法完美仿真在内部 FLASH 运行时的环境。另外，由于 STM32 无法直接从外部 SRAM 中启动且应用程序复制到外部 SRAM 的过程比较复杂(下载程序前需要使 STM32 能正常控制外部 SRAM)，所以在很少会在 STM32 的外部 SRAM 中调试程序。

### 49.2 STM32 的启动方式

在前面讲解的 STM32 启动代码章节了解到 CM-4 内核在离开复位状态后的工作过程如下，见图 49-1：

- (1) 从地址 0x00000000 处取出栈指针 MSP 的初始值，该值就是栈顶的地址。

- (2) 从地址 0x00000004 处取出程序指针 PC 的初始值，该值指向复位后应执行的第一条指令。

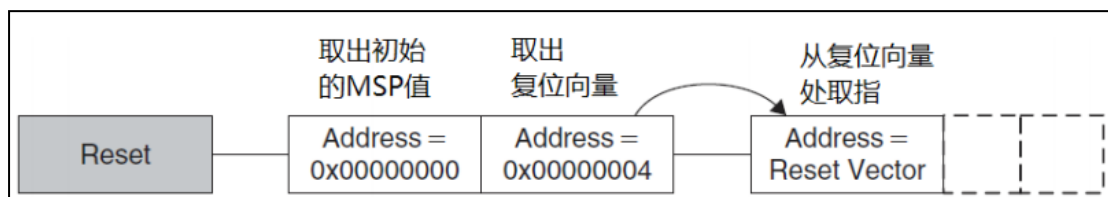


图 49-1 复位序列

上述过程由内核自动设置运行环境并执行主体程序，因此它被称为自举过程。

虽然内核是固定访问 0x00000000 和 0x00000004 地址的，但实际上这两个地址可以被重映射到其它地址空间。以 STM32F407 为例，根据芯片引出的 BOOT0 及 BOOT1 引脚的电平情况，这两个地址可以被映射到内部 FLASH、内部 SRAM 以及系统存储器中，不同的映射配置见表 49-1。

表 49-1 BOOT 引脚的不同设置对 0 地址的映射

| BOOT1 | BOOT0 | 映射到的存储器  | 0x00000000 地址映射到 | 0x00000004 地址映射到 |
|-------|-------|----------|------------------|------------------|
| x     | 0     | 内部 FLASH | 0x08000000       | 0x08000004       |
| 1     | 1     | 内部 SRAM  | 0x20000000       | 0x20000004       |
| 0     | 1     | 系统存储器    | 0x1FFF0000       | 0x1FFF0004       |

内核在离开复位状态后会从映射的地址中取值给栈指针 MSP 及程序指针 PC，然后执行指令，我们一般以存储器的类型来区分自举过程，例如内部 FLASH 启动方式、内部 SRAM 启动方式以及系统存储器启动方式。

#### (1) 内部 FLASH 启动方式

当芯片上电后采样到 BOOT0 引脚为低电平时，0x00000000 和 0x00000004 地址被映射到内部 FLASH 的首地址 0x08000000 和 0x08000004。因此，内核离开复位状态后，读取内部 FLASH 的 0x08000000 地址空间存储的内容，赋值给栈指针 MSP，作为栈顶地址，再读取内部 FLASH 的 0x08000004 地址空间存储的内容，赋值给程序指针 PC，作为将要执行的第一条指令所在的地址。具备这两个条件后，内核就可以开始从 PC 指向的地址中读取指令执行了。

#### (2) 内部 SRAM 启动方式

类似地，当芯片上电后采样到 BOOT0 和 BOOT1 引脚均为高电平时，0x00000000 和 0x00000004 地址被映射到内部 SRAM 的首地址 0x20000000 和 0x20000004，内核从 SRAM 空间获取内容进行自举。

在实际应用中，由启动文件 startup\_stm32f429\_439xx.s 决定了 0x00000000 和 0x00000004 地址存储什么内容，链接时，由分散加载文件(sct)决定这些内容的绝对地址，即分配到内部 FLASH 还是内部 SRAM。（下一小节将以实例讲解）

#### (3) 系统存储器启动方式

当芯片上电后采样到 BOOT0 引脚为高电平, BOOT1 为低电平时, 内核将从系统存储器的 0x1FFF0000 及 0x1FFF0004 获取 MSP 及 PC 值进行自举。系统存储器是一段特殊的空间, 用户不能访问, ST 公司在芯片出厂前就在系统存储器中固化了一段代码。因而使用系统存储器启动方式时, 内核会执行该代码, 该代码运行时, 会为 ISP 提供支持(In System Program), 如检测 USART1/3、CAN2 及 USB 通讯接口传输过来的信息, 并根据这些信息更新自己内部 FLASH 的内容, 达到升级产品应用程序的目的, 因此这种启动方式也称为 ISP 启动方式。

### 49.3 内部 FLASH 的启动过程

下面我们以最常规的内部 FLASH 启动方式来分析自举过程, 主要理解 MSP 和 PC 内容是怎样被存储到 0x08000000 和 0x08000004 这两个地址的。

见图 49-2, 这是 STM32F4 默认的启动文件的代码, 启动文件的开头定义了一个大小为 0x400 的栈空间, 且栈顶的地址使用标号 “\_\_initial\_sp” 来表示; 在图下方定义了一个名为 “Reset\_Handler” 的子程序, 它就是我们总是提到的在芯片启动后第一个执行的代码。在汇编语法中, 程序的名字和标号都包含它所在的地址, 因此, 我们的目标是把 “\_\_initial\_sp” 和 “Reset\_Handler” 赋值到 0x08000000 和 0x08000004 地址空间存储, 这样内核自举的时候就可以获得栈顶地址以及第一条要执行的指令了。在启动代码的中间部分, 使用了汇编关键字 “DCD” 把 “\_\_initial\_sp” 和 “Reset\_Handler” 定义到了最前面的地址空间。

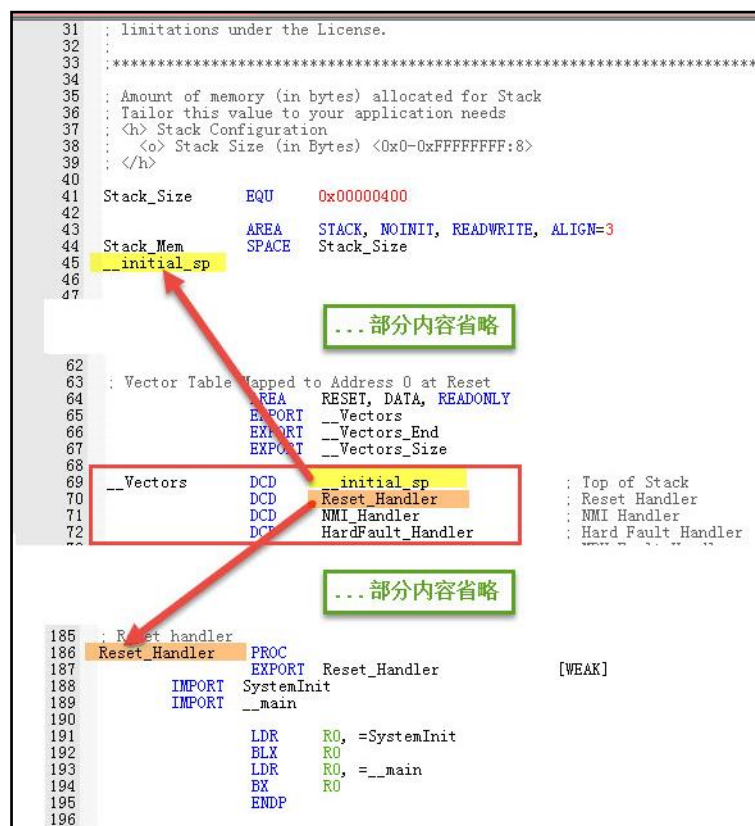


图 49-2 启动代码中存储的 MSP 及 PC 指针内容（启动文件）

startup\_stm32f40xx.s)

在启动文件中把设置栈顶及首条指令地址到了最前面的地址空间，但这并没有指定绝对地址，各种内容的绝对地址是由链接器根据分散加载文件(\*.sct)分配的，STM32F407ZGT6 型号的默认分散加载文件配置见代码清单 49-1。

代码清单 49-1 默认分散加载文件的空间配置

```
1 ; *****
2 ; *** Scatter-Loading Description File generated by uVision ***
3 ; *****
4
5 LR_IROM1 0x08000000 0x00100000 { ; load region size_region
6   ER_IROM1 0x08000000 0x00100000 { ; load address = execution address
7     *.o (RESET, +First)
8     *(InRoot$$Sections)
9     .ANY (+RO)
10  }
11  RW_IRAM1 0x20000000 UNINIT 0x00020000 { ; RW data
12    .ANY (+RW +ZI)
13  }
14 }
15
```

分散加载文件把加载区和执行区的首地址都设置为 0x08000000，正好是内部 FLASH 的首地址，因此汇编文件中定义的栈顶及首条指令地址会被存储到 0x08000000 和 0x08000004 的地址空间。

类似地，如果我们修改分散加载文件，把加载区和执行区的首地址设置为内部 SRAM 的首地址 0x20000000，那么栈顶和首条指令地址将会被存储到 0x20000000 和 0x20000004 的地址空间了。

为了进一步消除疑虑，我们可以查看反汇编代码及 map 文件信息来了解各个地址空间存储的内容，见图 49-3，这是多彩流水灯工程编译后的信息，它的启动文件及分散加载文件都按默认配置。其中反汇编代码是使用 fromelf 工具从 axf 文件生成的，具体过程可参考前面的章节了解。

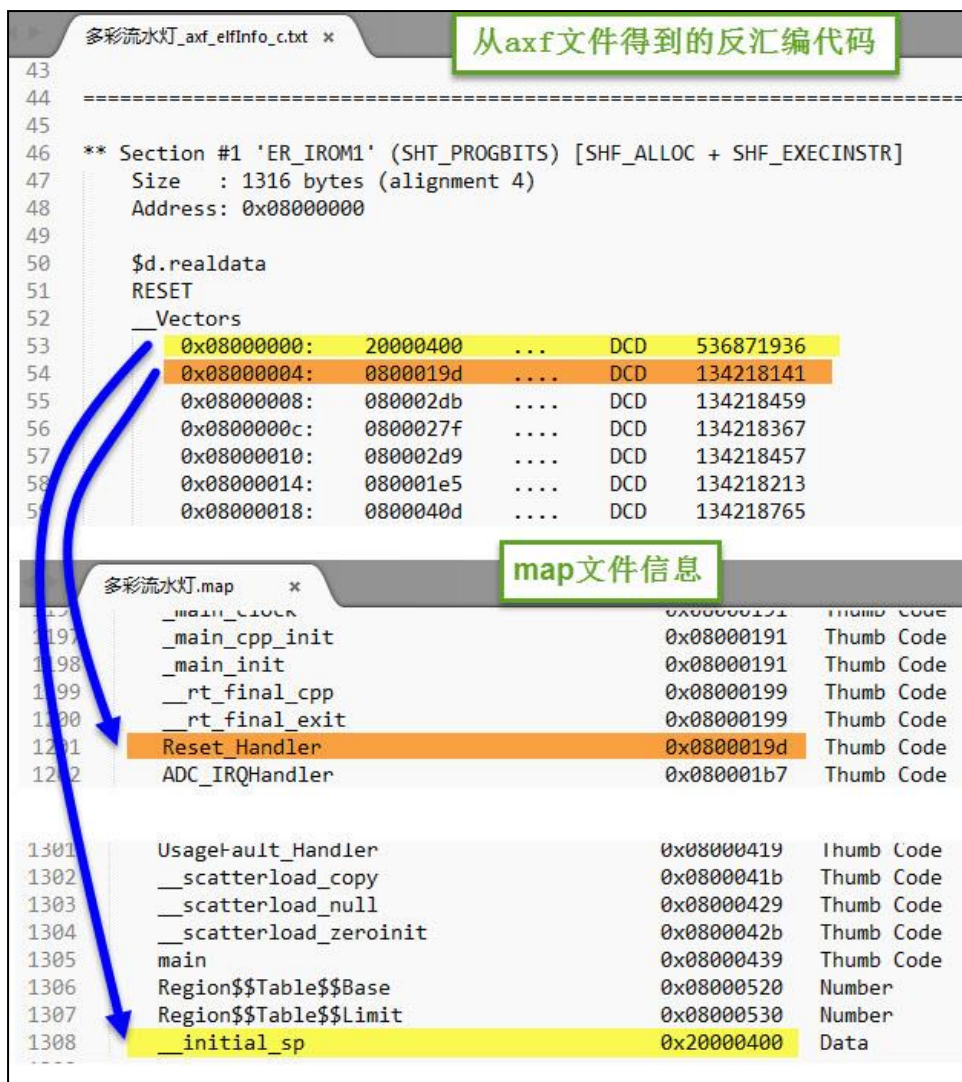


图 49-3 从反汇编代码及 map 文件查看存储器的内容

从反汇编代码可了解到，这个工程的 0x08000000 地址存储的值为 0x20000400，0x08000004 地址存储的值为 0x0800019d，查看 map 文件，这两个值正好是栈顶地址 \_\_initial\_sp 以及首条指令 Reset\_Handler 的地址。下载器会根据 axf 文件(bin、hex 类似)存储相应内容到内部 FLASH 中。

由此可知，BOOT0 为低电平时，内核复位后，从 0x08000000 读取到栈顶地址为 0x20000400，了解到子程序的栈空间范围，再从 0x08000004 读取到第一条指令的存储地址为 0x0800019d，于是跳转到该地址执行代码，即从 ResetHandler 开始运行，运行 SystemInit、\_\_main(包含分散加载代码)，最后跳转到 C 语言的 main 函数。

对比在内部 FLASH 中运行代码的过程，可了解到若希望在内部 SRAM 中调试代码，需要设置启动方式为从内部 SRAM 启动，修改分散加载文件控制代码空间到内部 SRAM 地址以及把生成程序下载到芯片的内部 SRAM 中。

## 49.4 实验：在内部 SRAM 中调试代码

本实验将演示如何设置工程选项实现在内部 SRAM 中调试代码，实验的示例代码名为“RAM 调试—多彩流水灯”，学习以下内容时请打开该工程来理解，它是从普通的多彩流水灯例程改造而来的。

### 49.4.1 硬件设计

在本节中使用到的流水灯硬件不再介绍，主要讲解与 SRAM 调试相关的硬件配置。在 SRAM 上调试程序，需要修改 STM32 芯片的启动方式，见图 49-4。

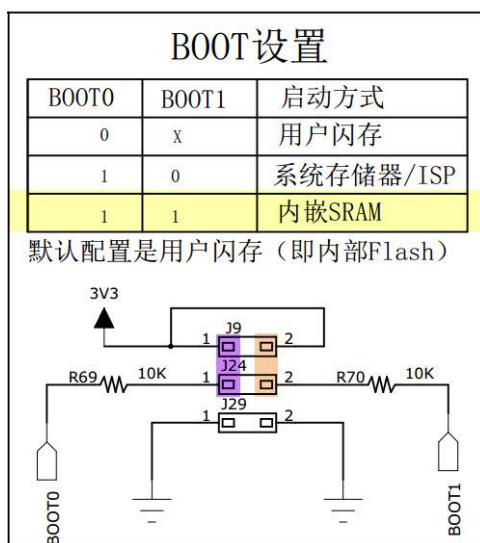


图 49-4 实验板的 boot 引脚配置

在我们的实验板左侧有引出 STM32 芯片的 BOOT0 和 BOOT1 引脚，可使用跳线帽设置它们的电平从而控制芯片的启动方式，它支持从内部 FLASH 启动、系统存储器启动以及内部 SRAM 启动方式。

本实验在 SRAM 中调试代码，因此把 BOOT0 和 BOOT1 引脚都使用跳线帽连接到 3.3V，使芯片从 SRAM 中启动。

### 49.4.2 软件设计

本实验的工程从普通的多彩流水灯工程改写而来，主要修改了分散加载文件及一些程序的下载选项。

#### 1. 主要步骤

- (1) 在原工程的基础上创建一个调试版本；
- (2) 修改分散加载文件，使链接器把代码分配到内部 SRAM 空间；
- (3) 添加宏修改 STM32 的向量表地址；
- (4) 修改仿真器和下载器的配置，使程序能通过下载器存储到内部 SRAM；

- (5) 根据使用情况选择是否需要使用仿真器命令脚本文件\*.ini;
- (6) 尝试给 SRAM 下载程序或仿真调试。

## 2. 创建工程的调试版本

由于在 SRAM 中运行的代码一般只是用于调试，调试完毕后，在实际生产环境中仍然使用在内部 FLASH 中运行的代码，因此我们希望能够便捷地在调试版和发布版代码之间切换。MDK 的“Manage Project Items”可实现这样的功能，使用它可管理多个不同配置的工程，见图 49-5，点击“Manage Project Items”按钮，在弹出对话框左侧的“Project Target”一栏包含了原工程的名字，如图中的原工程名为“多彩流水灯”，右侧是该工程包含的文件。为了便于调试，我们在左侧的“Project Target”一栏添加一个工程名，如图中输入“SRAM\_调试”，输入后点击 OK 即可，这个“SRAM\_调试”版本的工程会复制原“多彩流水灯”工程的配置，后面我们再进行修改。

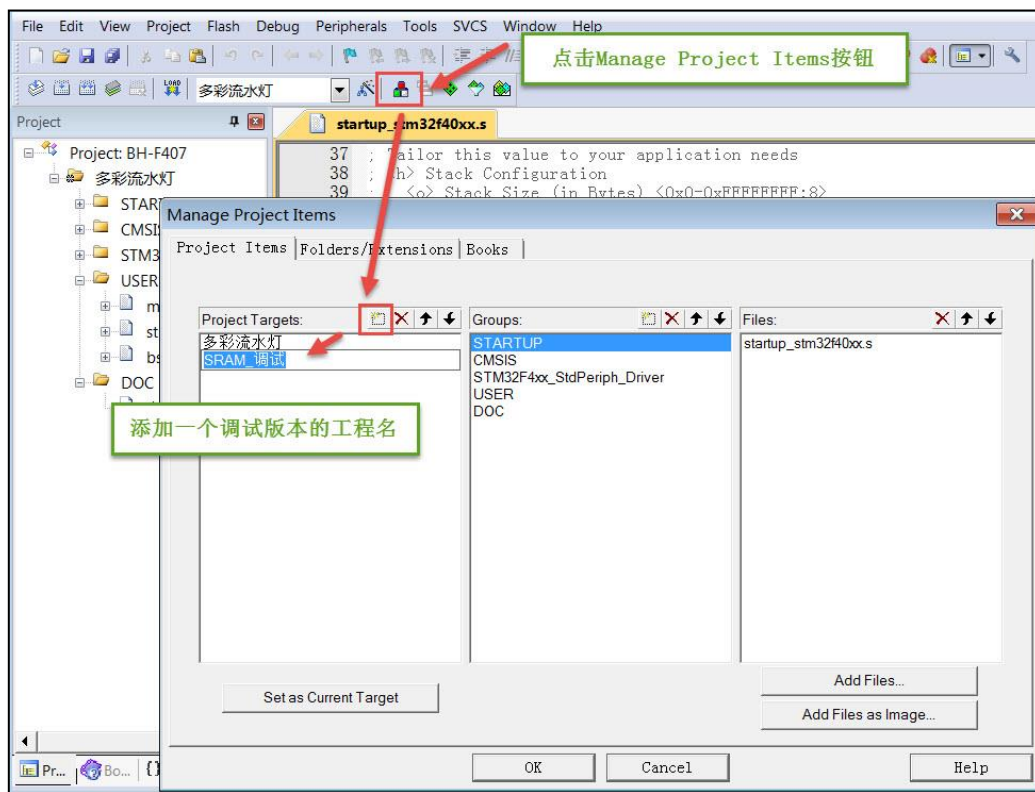


图 49-5 使用 Manage Project Items 添加一个工程配置

当需要切换工程版本时，点击 MDK 工程名的下拉菜单可选择目标工程，在不同的工程中，所有配置都是独立的，例如芯片型号、下载配置等等，但如果两个工程共用了同一个文件，对该文件的修改会同时影响两个工程，例如这两个工程都使用同一个 main 文件，我们在 main 文件修改代码，两个工程都会被修改。

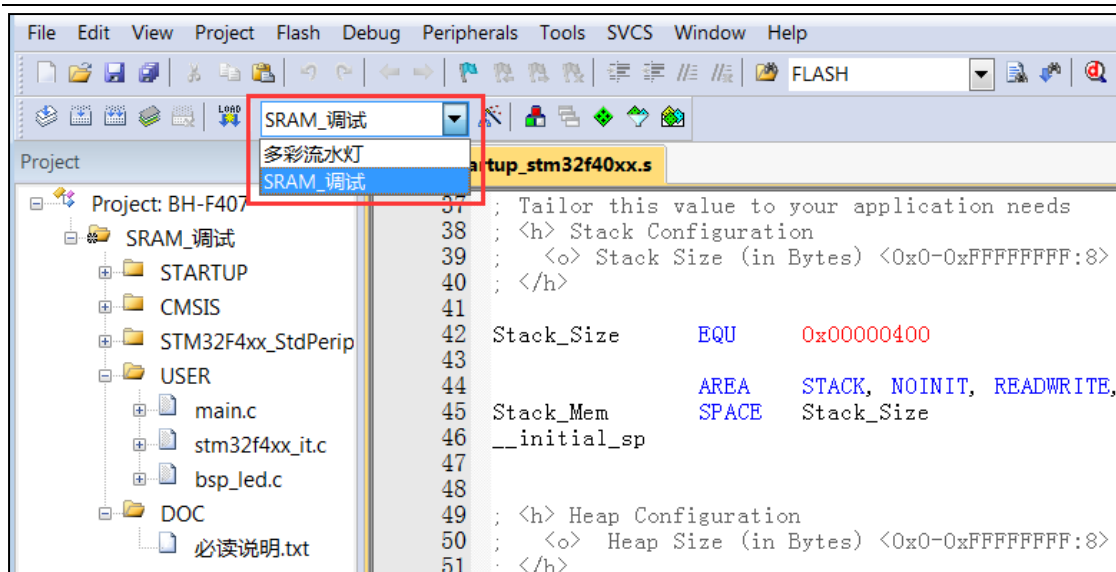


图 49-6 切换工程

在下面的教程中我们将切换到“SRAM\_调试”版本的工程，配置出一个代码会被存储到 SRAM 的多彩流水灯工程。

### 3. 配置分散加载文件

为方便讲解，本工程的分散加载只使用手动编辑的 sct 文件配置，不使用 MDK 的对话框选项配置，在“Options for Target->linker”的选项见图 49-7。

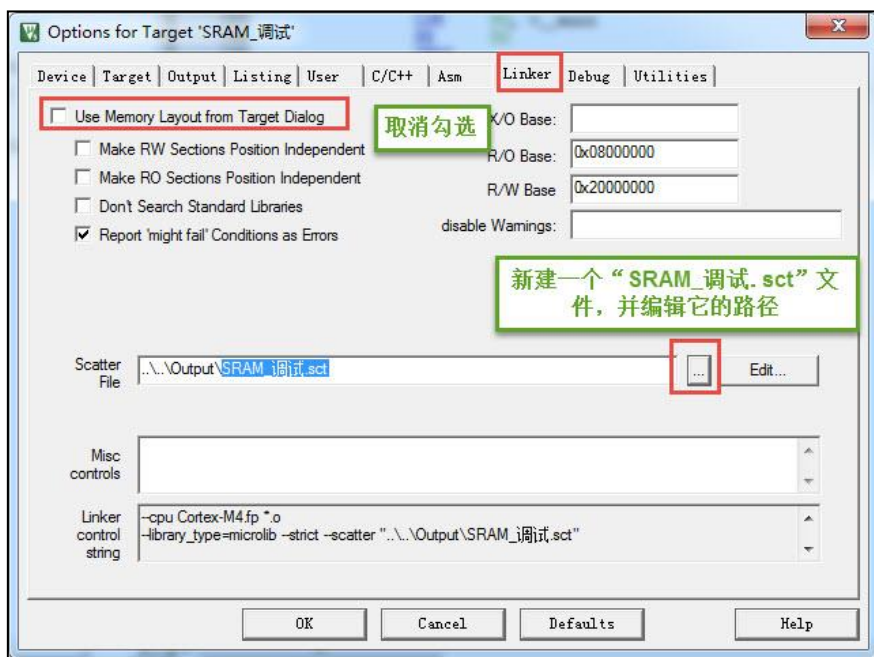


图 49-7 使用新建的“SRAM\_调试.sct”文件

为了防止“多彩流水灯”工程的分散加载文件被影响，我们在工程的 Output 路径下新建了一个名为“SRAM\_调试.sct”的文件，并在上图中把它配置为“SRAM\_调试”工程专用的分散加载文件，该文件的内容见代码清单 49-2，若不了解分散加载文件的使用，请参考前面的章节。

代码清单 49-2 分散加载文件配置(SRAM\_调试.sct)

```
1 ; *****
2 ; *** Scatter-Loading Description File generated by uVision ***
3 ; *****
4
5 LR_IROM1 0x20000000 0x00010000 { ; load region size_region
6   ER_IROM1 0x20000000 0x00010000 { ; load address = execution address
7     *.o (RESET, +First)
8     *(InRoot$$Sections)
9     .ANY (+RO)
10  }
11  RW_IRAM1 0x20010000 0x00010000 { ; RW data
12    .ANY (+RW +ZI)
13  }
14 }
15
```

在这个分散加载文件配置中，把原本分配到内部 FLASH 空间的加载域和执行域改到了以地址 0x20000000 开始的 64KB(0x00010000)空间，而 RW data 空间改到了以地址 0x20010000 开始的 64KB 空间 (0x00010000)。也就是说，它把 STM32 的内部 SRAM 分成了虚拟 ROM 区域以及 RW data 数据区域，链接器会根据它的配置给工程中的各种内容分配到 SRAM 地址。

在具体的应用中，虚拟 ROM 及 RW 区域的大小可根据自己的程序定制，配置完毕编译工程后可在 map 文件中查看具体的空间地址分配。

#### 4. 配置中断向量表

由于 startup\_stm32f40xx.s 文件中的启动代码不是指定到绝对地址的，经过它由链接器决定应存储到内部 FLASH 还是 SRAM，所以 SRAM 版本工程中的启动文件不需要作任何修改。

重点在于启动文件定义的中断向量表被存储到内部 FLASH 和内部 SRAM 时，这两种情况对内核的影响是不同的，内核会根据它的“向量表偏移寄存器 VTOR”配置来获取向量表，即中断服务函数的入口。VTOR 寄存器是由启动文件中 Reset\_Handle 中调用的库函数 SystemInit 配置的，见代码清单 49-3。

代码清单 49-3 SystemInit 函数(system\_stm32f4xx.c 文件)

```
1 /**
2  * @brief Setup the microcontroller system
3  * Initialize the Embedded Flash Interface, the PLL and update the
4  * SystemFrequency variable.
5  * @param None
6  * @retval None
7  */
8 void SystemInit(void)
9 {
10  /* ..其它代码部分省略 */
11
12  /* Configure the Vector Table location add offset address ----*/
13  #ifndef VECT_TAB_SRAM
14    SCB->VTOR = SRAM_BASE | VECT_TAB_OFFSET; /* 向量表存储在 SRAM */
15  #else
16    SCB->VTOR = FLASH_BASE | VECT_TAB_OFFSET; /* 向量表存储在内部 FLASH */
17  #endif
18 }
```

代码中根据是否存储宏定义 VECT\_TAB\_SRAM 来决定 VTOR 的配置，默认情况下代码中没有定义宏 VECT\_TAB\_SRAM，所以 VTOR 默认情况下指示向量表是存储在内部 FLASH 空间的。

由于本工程的分散加载文件配置，在启动文件中定义的中断向量表会被分配到 SRAM 空间，所以我们要定义这个宏，使得 SystemInit 函数修改 VTOR 寄存器，向内核指示向量表被存储到内部 SRAM 空间了，见图 49-8，在“Options for Target->c/c++->Define”框中输入宏“VECT\_TAB\_SRAM”，注意它与其它宏之间要使用英文逗号分隔开。

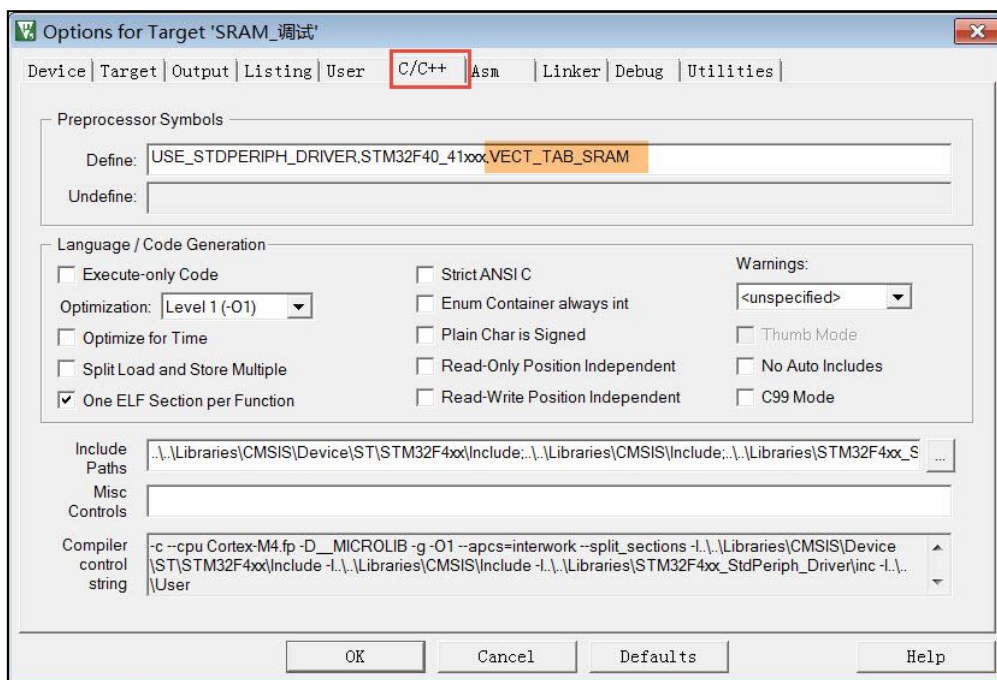


图 49-8 在 c/c++编译选项中加入宏 VECT\_TAB\_SRAM

配置完成后重新编译工程，即可生成存储到 SRAM 空间地址的代码指令。

## 5. 修改 FLASH 下载配置

得到 SRAM 版本的代码指令后，为了把它下载到芯片的 SRAM 中，还需要修改下载器的配置，见图 49-9，“Options for Target->Utilities->Settings”中的选项。

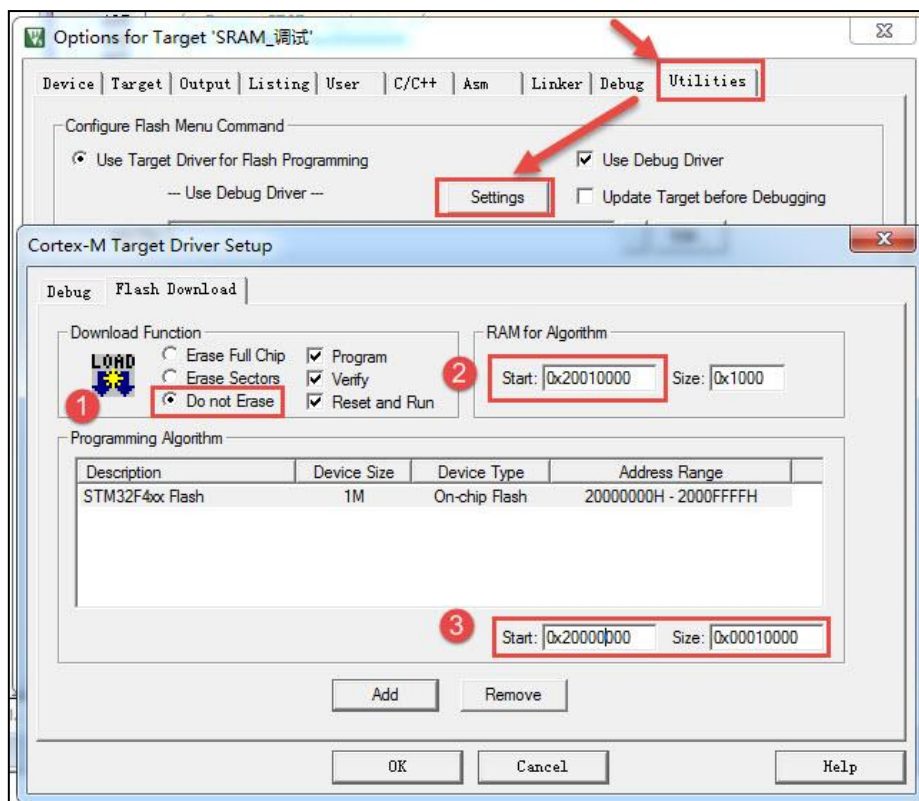


图 49-9 下载配置

这个配置对话框原本是用于设置芯片内部 FLASH 信息的，当我们点击 MDK 的 （下载、LOAD）按钮时，它会从此处加载配置然后下载程序到 FLASH 中，而在上图中我们把它的配置修改成下载到内部 SRAM 了，各个配置的解释如下：

- ❑ 把“Download Function”中的擦除选项配置为“Do not Erase”。这是因为数据写入到内部 SRAM 中不需要像 FLASH 那样先擦除后写入。在本工程中，如果我们不选择“Do not Erase”的话，会因为擦除过程导致下载出错。
- ❑ “RAM for Algorithm”一栏是指“编程算法”(Programming Algorithm)可使用的 RAM 空间，下载程序到 FLASH 时运行的编程算法需要使用 RAM 空间，在默认配置中它的首地址为 0x20000000，即内部 SRAM 的首地址，但由于我们的分散加载文件配置，0x20000000 地址开始的 64KB 实际为虚拟 ROM 空间，实际的 RAM 空间是从地址 0x20010000 开始的，所以这里把算法 RAM 首地址更改为本工程中实际作为 RAM 使用的地址。若编程算法使用的 RAM 地址与虚拟 ROM 空间地址重合的话，会导致下载出错。
- ❑ “Programming Algorithm”一栏中是设置内部 FLASH 的编程算法，编程算法主要描述了 FLASH 的地址、大小以及扇区等信息，MDK 根据这些信息把程序下载到芯片的 FLASH 中，不同的控制器芯片一般会有不同的编程算法。由于 MDK 没有内置 SRAM 的编程算法，所以我们直接在原来的基础上修改它的基地址和空间大小，把它改成虚拟 ROM 的空间信息。

从这个例子可了解到，这里的配置是跟我们的分散加载文件的实际 RAM 空间和虚拟 ROM 空间信息是一致的，若您的分散加载文件采用不同的配置，这个下载选项也要作出相应的修改，不能照抄本例子的空间信息。

这个配置是针对程序下载的，配置完成后点击 MDK 的  按钮（下载、LOAD），程序会被下载到 STM32 的内部 SRAM 中，复位后程序会正常运行（前提是 BOOT0 和 BOOT 要被设置为 SRAM 启动）。芯片掉电后这个存储在 SRAM 的程序会丢失，想恢复的话必须要重新下载程序。

## 6. 仿真器的配置

上面的下载配置使得程序能够加载到 SRAM 中全速运行，但作为 SRAM 版本的程序，其功能更着重于调试，也就是说我们希望它能支持平时使用  按钮（调试、debug）时进行的硬件在线调试、单步运行等功能。

要实现调试功能，还要在“Options for Target->Debug->Settings”中进行配置，见图 49-10。

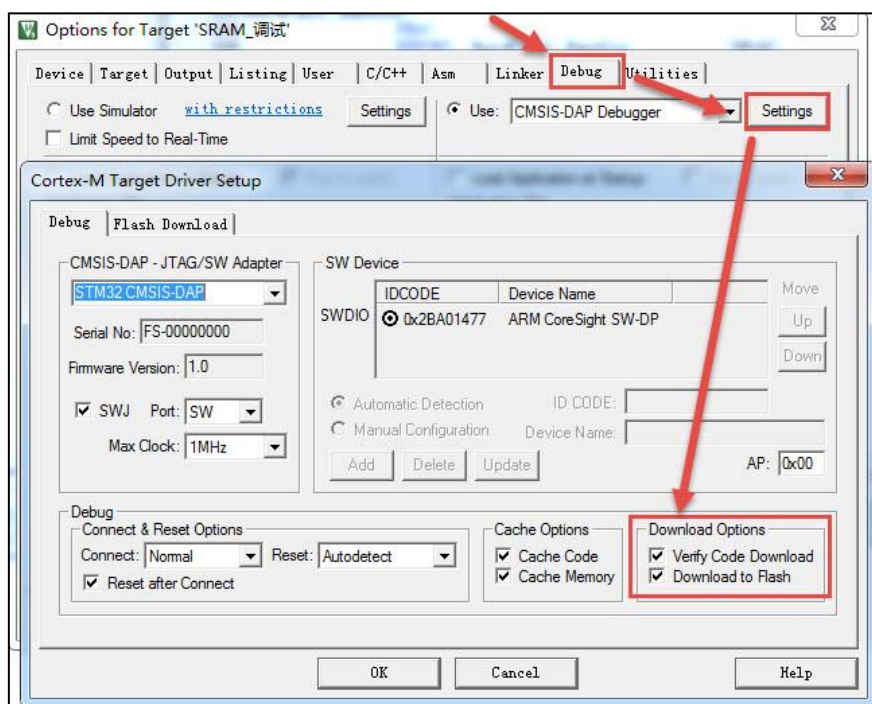


图 49-10 设置仿真前检查代码并下载程序到 FLASH 中

在图中我们需要勾选“Verify Code Download”及“Download to FLASH”选项，也就是说点击调试按钮后，本工程的程序会被下载到内部 SRAM 中，只有勾选了这两个选项才能正常仿真。（至于为什么 FLASH 版本的程序不需要勾选，不太清楚）

经过这样的配置后，硬件仿真时与平时内部 FLASH 版本的程序无异，支持软件复位、单步运行、全速运行以及查看各种变量值等（同样地，前提是 BOOT0 和 BOOT 要被设置为 SRAM 启动）。

## 7. 不需要修改 BOOT 引脚的仿真配置

假如您使用的硬件平台中 BOOT0 和 BOOT1 引脚电平已被固定，设置为内部 FLASH 启动，不方便改成 SRAM 方式，可以使用如下方法配置调试选项实现在 SRAM 调试：

- (1) 与上述步骤一样，勾选“Verify Code Download”及“Download to FLASH”选项；
- (2) 见图 49-11，在“Options for Target->Debug”对话框中取消勾选“Load Application at startup”选项。点击“Initialization File”文本框右侧的文件浏览按钮，在弹出的对话框中新建一个名为“Debug\_RAM.ini”的文件；

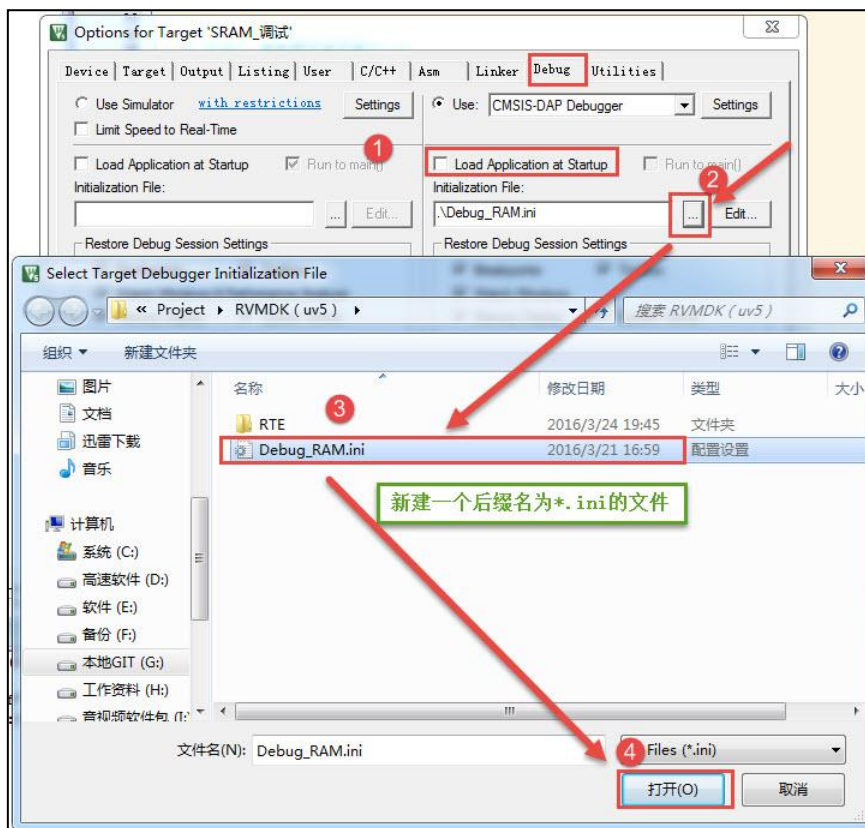


图 49-11 新建一个 ini 文件

- (3) 在 Debug\_RAM.ini 文件中输入如代码清单 49-4 中的内容。

代码清单 49-4 Debug\_RAM.ini 文件内容

```
1 /*****  
2 /* Debug_RAM.ini: Initialization File for Debugging from Internal RAM */  
3 /*****  
4 /* This file is part of the uVision/ARM development tools. */  
5 /* Copyright (c) 2005-2014 Keil Software. All rights reserved. */  
6 /* This software may only be used under the terms of a valid, current, */  
7 /* end user licence from KEIL for a compatible version of KEIL software */  
8 /*development tools. Nothing else gives you the right to use this software */  
9 /*****  
10  
11 FUNC void Setup (void) {  
12     SP = _RDWORD(0x20000000); // 设置栈指针 SP, 把 0x20000000 地址中的内容赋值到 SP。  
13     PC = _RDWORD(0x20000004); // 设置程序指针 PC, 把 0x20000004 地址中的内容赋值到 PC。  
14     XPSR = 0x01000000; // 设置状态寄存器指针 xPSR  
15     _WDWORD(0xE000ED08, 0x20000000); // Setup Vector Table Offset Register  
16 }
```

```
17
18 LOAD %L INCREMENTAL           // 下载 axf 文件到 RAM
19 Setup();                     // 调用上面定义的 setup 函数设置运行环境
20
21 //g, main //跳转到 main 函数，本示例调试时不需要从 main 函数执行，注释掉了，程序从启动代码开始执行
```

上述配置过程是控制 MDK 执行仿真器的脚本文件 Debug\_RAM.ini，而该脚本文件在下载了程序到 SRAM 后，初始化了 SP 指针(即 MSP)和 PC 指针分别指向了 0x20000000 和 0x20000004，这样的操作等效于从 SRAM 复位。

有了这样的配置，即使 BOOT0 和 BOOT1 引脚不设置为 SRAM 启动也能正常仿真了，但点击下载按钮把程序下载到 SRAM 然后按复位是不能全速运行的(这种运行方式脱离了仿真器的控制，SP 和 PC 指针无法被初始化指向 SRAM)。

上述 Debug\_RAM.ini 文件是从 STM32F4 的 MDK 芯片包里复制过来的，若您感兴趣可到 MDK 安装目录搜索该文件名，该文件的语法可以从 MDK 的帮助手册的“μVision User's Guide->Debug Commands”章节学习。